Policy Bank: Development-Selected Routing of Public Robot Policies for BEHAVIOR-1K

Team KANR technical report, 2026 BEHAVIOR Challenge

Keshav Balakrishna Abhijay Gangerapu
Rithik Satarla
Team KANR (Independent)
https://github.com/FungousLand1941/policy-bank

October 2026

Abstract

The 2026 BEHAVIOR Challenge asks one system to complete 100 long-horizon household tasks in simulation. Rather than train a single generalist, we assemble a bank of ten public and lightly fine-tuned vision-language-action checkpoints and route each task to one of them. Routing uses only development instances: a hierarchical empirical-Bayes model combines public reference results, policy-level estimates paired against a reference policy, and our ledger of 1,146 development rollouts, and a conservative rule keeps the incumbent policy unless a challenger also beats it on shared instances. The frozen route was evaluated once on the ten public reporting instances of every task. It reaches $Q = 0.156$ (95% CI 0.144–0.168) over 1,000 episodes, earning partial credit on 71 tasks but full success in only 37 episodes. Development estimates rank tasks well (Spearman 0.68) yet are optimistic by 0.050 on average, and rollout noise (s.d. 0.20 within a task) means telling apart two policies that differ by 0.05 takes about 251 rollouts each. We release the route, all 1,000 metric files, the development ledger, a reimplementation of the selector that reproduces 48 of 50 stage-2 decisions, and a fail-stop serving layer for hidden evaluation.

1 Introduction

BEHAVIOR-1K [7, 8] defines everyday household activities in realistic simulation, and its 2026 challenge [13] scores a single submission on 100 of them. Each task is long: the mean human demonstration runs from about one to fifteen minutes, and an episode ends when the goal is reached or when the robot has used 1.5 times the mean demonstration length. Partial credit counts the fraction of goal conditions satisfied, and the challenge score Q averages it over tasks.

By the 2026 challenge several strong public checkpoints existed, each built on the $\pi_0/\pi_{0.5}$ vision-language-action family [3, 9, 10]: the Comet team's 2025 models [1], the four checkpoints of the 2025 winning solution [5], continuations of these on the 2026 demonstrations, and community specialists fine-tuned for a handful of tasks. They differ sharply by task. A checkpoint that wins one task may score zero on the next. With a fixed compute budget, the question we study is practical: *which checkpoint should run each task, decided only from development data, without fooling ourselves with noise?*

This is an algorithm-selection problem [11] with three complications. First, evidence is scarce: a single rollout runs for up to 22 simulated minutes, and longer in wall-clock time, so most (task, policy) pairs get one to three development rollouts. Second, rollouts are noisy: the same policy on the same task scores 0 on one instance and 1 on the next. Third, choosing the best-looking option from noisy estimates is optimistically biased (the optimizer's curse [12]). Our approach treats routing as shrinkage estimation [4]. Every (task, policy) pair gets an informative prior, development rollouts update it, and a policy displaces the incumbent only when several conservative conditions agree.

Contributions.

1. A *policy bank* of ten checkpoints, each served with its native adapter, and a frozen per-task route chosen without looking at reported instances (Sections 4–5).
2. A development-selection method: transfer priors from public reference results, paired policy-level priors where none exist, conjugate updates, and strict switching rules (Section 5).
3. An audited evaluation: $Q = 0.156$ [0.144, 0.168] on 1,000 unchanged episodes, with score-blind duplicate handling and every deviation disclosed (Sections 6–7).
4. An analysis of calibration, noise and statistical power that explains why per-task routing gains are hard to verify at challenge scale (Section 8).
5. A fail-stop serving layer for hidden evaluation and an open release of code, data and route (Section 9).

2 Related work

Vision-language-action policies. π_0 [3] attaches a flow-matching action expert to a PaliGemma vision-language backbone [2], and $\pi_{0.5}$ [10] adds co-training for open-world generalization. All checkpoints in our bank descend from this line through OpenPI [9].

BEHAVIOR solutions. The 2025 winning solution [5] adapted $\pi_{0.5}$ with correlated-noise flow matching, stage tracking and task-specific correction rules, reaching $Q{\approx}0.26$ on 50 tasks. Openpi Comet [1], second in 2025, studied pre-training and post-training recipes at scale. We train almost nothing ourselves. Our contribution is to select among such models and to measure the result carefully.

Selection under noise. Per-instance algorithm selection [11] picks the best solver for each input. Empirical Bayes and Stein shrinkage [4] improve many noisy means at once by borrowing strength across them, and the optimizer's curse [12] predicts that the chosen option will disappoint on average. Our results show this effect directly (Section 8.1).

3 Problem setup

Tasks and scoring. Each of the 100 tasks t has a goal written in BDDL. An episode ends at success or when the simulator step count reaches 1.5 times the mean demonstration length for t. Its score $q \in [0, 1]$ is the fraction of goal conditions satisfied at the end, with $q = 1$ exactly at success. With $I = \{301, \ldots, 310\}$ the reporting instances,

$$Q = \frac{1}{100} \sum_{t=1}^{100} \frac{1}{|I|} \sum_{i \in I} q_{t,i}, \tag{1}$$

where a missing cell counts as zero. Instances 311–320 are reserved for development; the organizers evaluate submitted containers on hidden instances 321–340.

Robot and interface. The robot is a wheeled bimanual R1Pro. A policy receives head and wrist RGB-D images, a 61-dimensional proprioceptive vector and the task identity. It returns 23-dimensional actions over a websocket, optionally several per query. Hidden evaluation requires an average of at least one simulator frame per second including inference, at most 600 s per query, and a single 24 GB GPU per policy.

Rules we imposed on ourselves. Policies use only onboard sensors at test time. Every routing decision uses development instances only, every reported cell is run once, and a route is frozen and hashed before reported evaluation of the affected tasks begins.

4 The policy bank

Table 1 lists the ten routed checkpoints with their exact sources. They fall into three families. **Comet** ($\pi_{0.5}$, 32-step action chunks): the released `pt50` and `pt12` checkpoints [1, 14]; a public continuation of `pt50` on the 2026 demonstrations by ymetz (step 9999); and `s150997`, our own continued training of `pt50` on the

Table 1: The routed checkpoints. Revisions are Hugging Face commit prefixes. No weights are redistributed with this report.

Policy	Source (repository/subfolder)	Revision
`pt50`	`sunshk/openpi_comet/pi05-b1kpt50-cs32`	`61739ff`
`pt12`	`sunshk/openpi_comet/pi05-b1kpt12-cs32`	`61739ff`
`s150997`	`this work (continued training of pt50)`	`step 150997`
`comet@ymetz9999`	`ymetz/pi05-b1k-pt50-steps-public/steps-cont-9999`	`20e80d2`
`winner1`	`IliaLarchenko/behavior_submission/checkpoint_1`	`89545bc`
`winner2`	`IliaLarchenko/behavior_submission/checkpoint_2`	`89545bc`
`winner3`	`IliaLarchenko/behavior_submission/checkpoint_3`	`89545bc`
`winner4`	`IliaLarchenko/behavior_submission/checkpoint_4`	`89545bc`
`w100@j_jl_4task100`	`JackLiu0406/b1k-checkpoints/pi_behavior_100t_meta_ft_4task`	`ec1844b`
`w100@b_amy_t90`	`amyjia1212/b1k-2026-t90-91-93-96-97/step_099999`	`7519e44`

2026 demonstrations of all 100 tasks. **Winner** (re-planning every 20 steps): the four checkpoints of the 2025 winning solution [5, 6], served with their native wrapper and correction rules. **Winner-format specialists**: two community models fine-tuned for a few 2026 tasks, one covering tasks 54, 56, 77 and 98 and one released for tasks 90, 91, 93, 96 and 97.

Every checkpoint runs behind its own native adapter: prompts, image preprocessing, normalization statistics, action representation and replay horizon are those of its publisher. We do not merge weights or ensemble actions; the bank is a lookup table from task to checkpoint. Our development ledger also covers 36 policy labels in total, including other checkpoint steps, a third-party 100-task model, action-search variants, time compression, gripper-correction add-ons and a modified robot configuration. None of these entered the route.

5 Development-selected routing

5.1 Model

Let θ_{tp} be the expected score of policy p on task t. We place a Gaussian prior $\theta_{tp} \sim \mathcal{N}(\mu_{tp}, \tau_{tp}^2)$ and treat each development rollout as $y_{tpk} \sim \mathcal{N}(\theta_{tp}, \sigma_p^2)$. With n rollouts of mean $\bar{y}$, the posterior is

$$\theta_{tp} \mid y \sim \mathcal{N}\left(\frac{\mu_{tp}/\tau_{tp}^2 + n\bar{y}/\sigma_p^2}{1/\tau_{tp}^2 + n/\sigma_p^2},\ \left(\frac{1}{\tau_{tp}^2} + \frac{n}{\sigma_p^2}\right)^{-1} \right). \tag{2}$$

The rollout variance σ_p^2 is the pooled within-task variance of p's development rollouts, shrunk toward a default with a few pseudo-degrees of freedom. Rollout scores are bounded and often bimodal, so the Gaussian is an approximation. Its job is to weigh a prior against a handful of rollouts, not to model q exactly.

5.2 Priors where public reference results exist (tasks 0–49)

For the first 50 tasks, the 2025 challenge and its participants published per-task results for the Comet and winner checkpoints. These results come from a different simulator setup and different instances, so we scale them by a transfer factor γ:

$$\mu_{tp} = \gamma_f \, r_{tp}, \qquad \tau_{tp}^2 = (r_{tp}\, s_\gamma)^2 + b^2, \tag{3}$$

where r_{tp} is the reference score, s_γ the factor's uncertainty and $b \approx 0.09$ a floor for task-specific deviations. For `pt50`, γ is estimated as the ratio of our development means to the reference over the other tasks (leave-one-task-out), giving 0.55 ± 0.09. For the winner family we fixed $\gamma = 0.81 \pm 0.10$ before stage 2; an empirical-Bayes estimate from 31 winner development rollouts gave 0.69 ± 0.11.

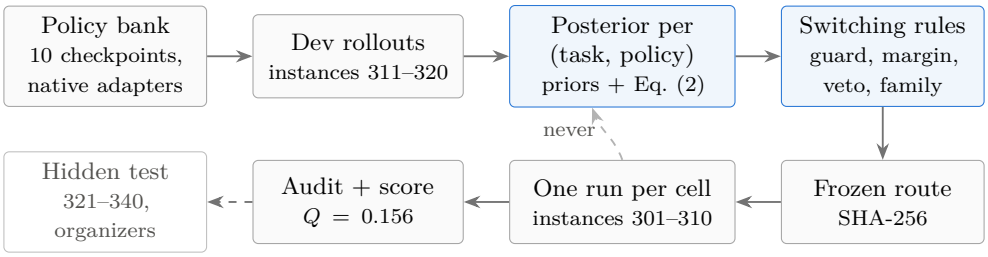


Figure 1: Development-selected routing. Reported instances (301–310) are never an input to a routing decision. The route is frozen and hashed before reported evaluation.

5.3 Priors without reference results (tasks 50–99)

Tasks 50–99 are new in 2026, so no public per-task results exist. We estimate a *policy level* instead. The reference policy `s150997` has development rollouts on almost every task, so its level ℓ_{ref} is the mean of its per-task means over the other tasks. A candidate's level is the reference level plus its mean paired difference on the tasks both have run:

$$\ell_p = \ell_{\text{ref}} + \frac{1}{|S_p|} \sum_{u \in S_p} \big(\bar{y}_{up} - \bar{y}_{u,\text{ref}}\big), \qquad S_p = \{u \neq t : \text{ both ran } u\}. \tag{4}$$

Pairing removes much of the between-task variation, because candidates are compared on the same tasks. Each level is combined with its family's other members, or with a weak hyperprior $\mathcal{N}(0.1, 0.1^2)$, and widened by a between-task standard deviation of 0.1 to give (μ_{tp}, τ_{tp}). For example, the ymetz continuation has a level of 0.184 ± 0.048 and a task prior of 0.184 ± 0.111. One development rollout scoring 0 moves this prior to a posterior mean of 0.160, as Eq. (2) predicts.

5.4 Switching rules

A posterior ranking alone would chase noise, so a challenger must pass every rule below:

1. **Own evidence**: at least two of its own development rollouts on the task (tasks 0–49).
2. **Development guard**: it must also beat the incumbent on the development instances both ran (at least two shared instances), not only in posterior mean.
3. **Margin**: posterior gain of at least 0.01 over the incumbent.
4. **Veto**: it is rejected if it scored 0 on every instance where the incumbent scored above 0.1.
5. **Scope**: a specialist is a candidate only on tasks where it has its own development rollouts, and only if it has at least two rollouts in total.
6. **One checkpoint per family**: when two checkpoints of one family win different tasks, the one with the larger summed posterior gain takes both. This keeps the number of serving images small.

On tasks 0–49 every challenger must pass all rules. On tasks 50–99 the incumbent is the weak reference policy and the highest eligible posterior wins after the veto, scope and family rules.

5.5 Two-stage freeze

Evaluation compute was the binding constraint, so routing was frozen in two stages (Figure 1). Stage 1 fixed the 37 tasks whose choice was already settled by priors and early development evidence, and their reported runs began on 11 October. The other 63 tasks were deferred while development rollouts accumulated. Their route was written from 860 development rollouts and frozen at 12:54 UTC on 13 October. Every reported episode of these tasks ran under its stage-2 entry; the earliest finished at 13:10 UTC. Each deferred task also carried a fallback policy, to be used if its stage-2 entry had not arrived by a deadline. The route tables are identified by SHA-256 in the release, and every reported episode was produced by its task's routed policy (checked in the test suite).

Table 2: Headline results on the reporting instances (Isaac Sim 6.0.1). The confidence interval resamples instances within each task (10,000 bootstrap draws). This is a self-evaluated public score, not a hidden-test score.

Challenge score Q (95% CI)	0.156 (0.144–0.168)
Episodes / tasks	1,000 / 100
Full successes	37 (3.7%) in 15 tasks
Tasks with partial credit ($\bar{q} > 0$)	71
Tasks scoring zero on all ten instances	29
Episodes with $q > 0$	364

6 Evaluation protocol

Infrastructure. Reported and development runs used TACC Stampede3 nodes with two NVIDIA RTX PRO 6000 Blackwell GPUs, running Isaac Sim 6.0.1 inside the organizers' container image. Isaac Sim 5.1, the version prescribed for hidden evaluation, crashed with the cluster's GPU driver; the organizers allowed Isaac Sim 6 for public results. Each simulator connects to its own policy server, because one native server keeps a single action plan for all connections. Camera inputs use the full-resolution RGB-D wrapper. A paired development comparison over 38 cells found 224-pixel inputs worse by 0.061 on average, so we kept full resolution. Over the project we used roughly 1,500 documented allocated GPU-hours, including idle and failed allocations.

Integrity rules. The reporting set contains exactly one unchanged metrics file per (task, instance), 1,000 in total, each identified by SHA-256. Infrastructure retries produced two completed duplicate cells. In both we kept the attempt that *finished first*, regardless of its score: `spraying_for_bugs`/307 (kept 0.5, later 0.0) and `cook_brussels_sprouts`/303 (kept 0.0, later 0.0). Logs show the later attempt started after the kept one ended. No development rerun or later experiment was merged into the reporting set.

Execution details. Comet-family policies replay 32 actions per query and winner-format policies 20. In the reporting set, 716 episodes ran with replay 32, 277 with replay 20, and 7 came from a recovery pass that replays 0 actions (one query per control step). We report those seven unchanged. The copy of the organizers' scorer bundled with the simulator divides by a 20-instance public pool and returns 0.078 on our files. The published reporting contract uses ten instances, and the same unmodified functions with that denominator give 0.15605. We changed neither the scorer nor the files.

7 Results

Table 2 summarizes the run. The score is mostly partial credit: 364 episodes satisfy at least one goal condition, but only 37 finish their task. All 963 unsuccessful episodes ran to the step cap. None ended early for any other reason.

Figure 2 shows the per-task picture. The highest-scoring tasks are served by the four-task specialist and the strongest Comet and winner checkpoints (Table 3). Twenty-nine tasks score zero on all ten instances. Their demonstrations are not longer than average (mean 319 s against 352 s over all tasks). On 23 of them no development rollout of any policy scored above zero either. On the other six, at least one development rollout made progress, in four cases a full success, which shows how far a single development rollout can mislead.

Per policy. Table 4 and Figure 3 break the score down by policy. The ymetz continuation of `pt50` carries 42 of the 50 new tasks and contributes the most task points. Its per-task average (0.111) is modest, but it beat our own continued training on development pairs. The four-task specialist is the standout: 0.581 on its four tasks, eleven of the 37 successes, and well above its expectation. The two `pt12` tasks and `winner4`'s microwave-popcorn task also score high. Our own checkpoint `s150997` kept only three tasks and earned almost nothing on them, which is consistent with its low development level.

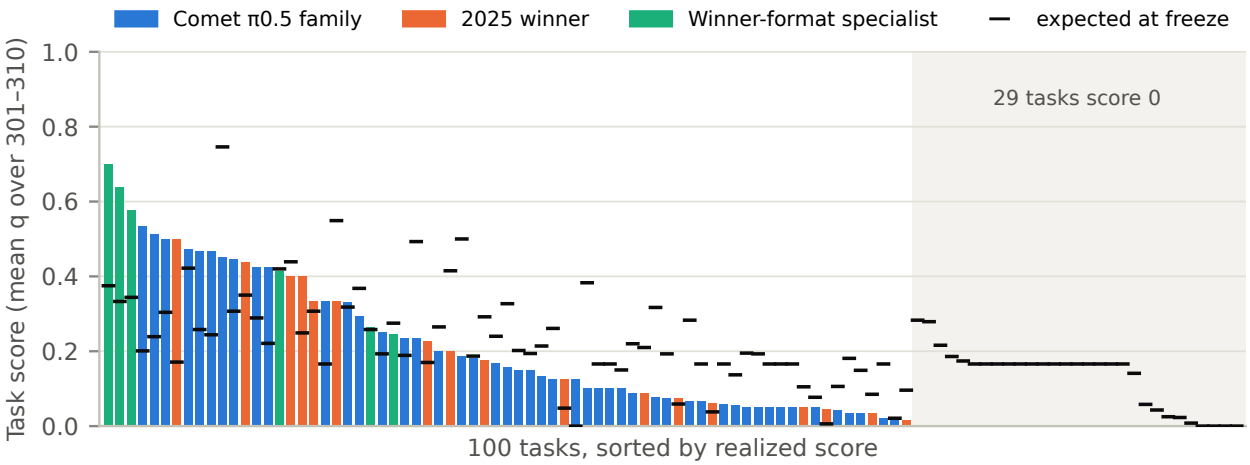


Figure 2: Realized score per task (bars, colored by policy family) against the posterior expectation at freeze (ticks), sorted by realized score. Twenty-nine tasks score zero on all ten instances.

Table 3: The ten highest-scoring tasks.

ID	Task	Policy	Expected	Realized	Successes
77	installing a modem	`w100@j_jl_4task100`	0.375	0.700	6
54	putting away toys	`w100@j_jl_4task100`	0.333	0.637	1
56	make rose centerpieces	`w100@j_jl_4task100`	0.344	0.575	2
80	clean up broken glass	`comet@ymetz9999`	0.201	0.533	2
24	storing food	`pt50`	0.239	0.512	0
0	turning on radio	`pt12`	0.304	0.500	5
31	clean boxing gloves	`winner3`	0.171	0.500	5
21	collecting childrens toys	`pt50`	0.422	0.471	0
58	stacking wood	`comet@ymetz9999`	0.244	0.467	2
87	collecting aluminum cans	`comet@ymetz9999`	0.258	0.467	0

Per scene. Scores range from 0.30 on the five `hotel_suite_large` tasks to 0.075 on the ten upper-floor tasks of `house_double_floor` (Table 5). Mean demonstration length is only weakly related to task score (Spearman 0.11).

8 Analysis

8.1 Calibration and the optimizer's curse

Figure 4 compares each task's posterior expectation at freeze with its realized score. The ranking holds up: Spearman 0.68 and Pearson 0.60 across the 100 tasks. The level does not. The mean expectation was 0.206, the mean realization 0.156. This is the optimizer's curse [12]: we chose, per task, the policy whose estimate looked best, so on average its estimate was too high. Shrinkage reduces the bias but cannot remove it with this little data.

The bias differs by stage. Stage-1 tasks realized 0.115 against an expected 0.188 (ratio 0.61). Stage-2 tasks realized 0.180 against 0.217 (ratio 0.83). Stage-1 priors leaned on 2025 reference results transferred to a new simulator and new instances, so they carried the most optimism. Stage-2 priors were built from our own paired development rollouts, under the same setup as the reported run.

Table 4: Per-policy outcome. "Expected" is the mean posterior expectation over the policy's routed tasks at freeze. "Task points" is the sum of its tasks' scores, so the column sums to 100 Q.

Policy	Family	Replay	Tasks	Expected	Realized	Successes	Task points
`comet@ymetz9999`	Comet	32	42	0.188	0.111	5	4.65
`pt50`	Comet	32	25	0.230	0.145	5	3.63
`w100@j_jl_4task100`	Specialist	20	4	0.368	0.581	11	2.32
`winner2`	Winner	20	5	0.355	0.292	1	1.46
`winner1`	Winner	20	11	0.108	0.094	1	1.04
`pt12`	Comet	32	2	0.305	0.472	5	0.94
`winner3`	Winner	20	5	0.137	0.119	5	0.59
`w100@b_amy_t90`	Specialist	20	2	0.267	0.255	0	0.51
`winner4`	Winner	20	1	0.439	0.400	4	0.40
`s150997`	Comet	32	3	0.098	0.017	0	0.05

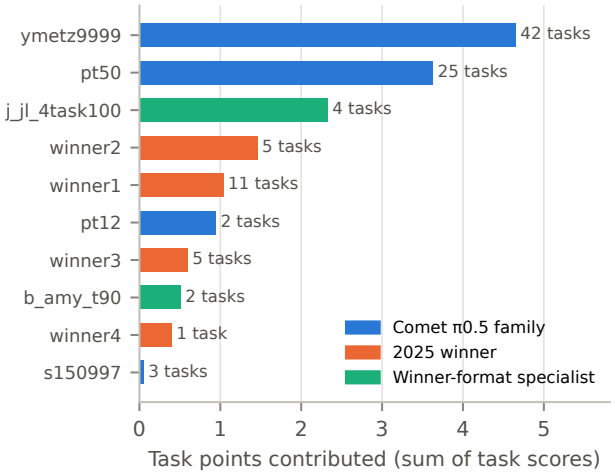


Figure 3: Contribution of each routed policy to Q, in task points.

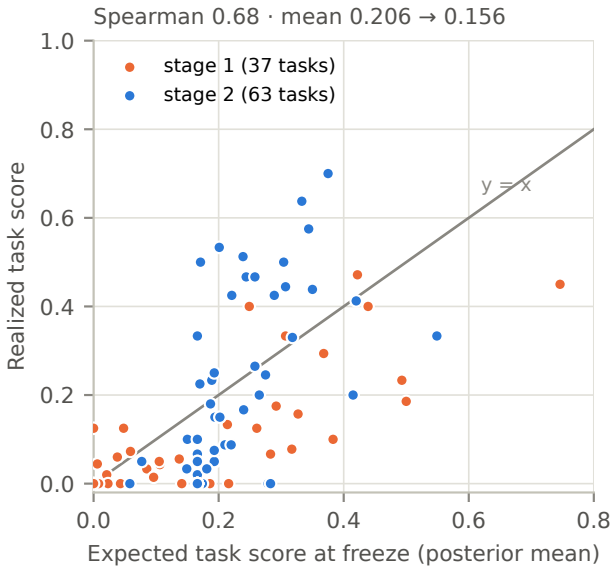


Figure 4: Expected (posterior at freeze) versus realized task score. Points below the diagonal disappointed.

8.2 Rollout noise and statistical power

Within a task, reported episode scores have a standard deviation of 0.20, and 642 of 1,000 episodes score below 0.1 (Figure 5). Telling apart two policies whose true task scores differ by 0.05 takes about 251 rollouts per policy at 80% power; a difference of 0.10 takes 63 and a difference of 0.20 takes 16. Our whole ledger has 1,146 rollouts over 36 policy labels. Of the 404 (task, policy) cells eligible for selection, 198 have a single rollout and 91% have at most three, and 57% of all development rollouts used instance 313. In this regime only large differences can be resolved per task. The priors and paired levels in Section 5 carry most of the information, and the switching rules exist to stop single lucky rollouts from changing the route. Before the deadline we estimated that all 54 remaining per-task switch proposals together were worth about +0.007 Q, too little to justify a rerun.

8.3 Reproducing the selection

Our package contains a clean reimplementation of the selector (`policybank.selection`). Run on the released development ledger, restricted to rollouts finished before the stage-2 freeze (411 rollouts on tasks 50–99), it reproduces 48 of the 50 stage-2 decisions for those tasks. In both disagreements (tasks 70 and 87) the reimplementation keeps the reference policy where the frozen route chose the ymetz continuation. The frozen

Table 5: Mean task score by scene.

Scene	Tasks	Mean task score
`hotel_suite_large`	5	0.302
`restaurant_diner`	8	0.198
`house_single_floor`	34	0.170
`Rs_int`	6	0.156
`house_double_floor_lower`	32	0.137
`office_cubicles_right`	5	0.128
`house_double_floor_upper`	10	0.075

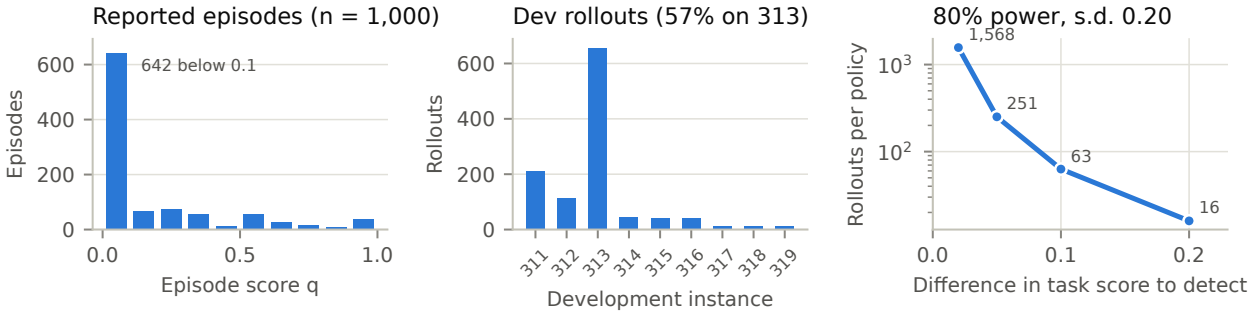


Figure 5: Left: episode scores pile up at zero. Middle: development rollouts concentrated on one instance. Right: rollouts per policy needed to detect a given difference in task score at 80% power ($\alpha = 0.05$, unpaired), using the within-task standard deviation of the reported run.

route came from the original pipeline, whose development table and filtering differ slightly from the released ledger. We do not reproduce tasks 0–49 because their priors depend on third-party reference tables that we do not redistribute.

8.4 What did not make the route

Several ideas were tested on development instances and rejected, or never reached a decisive comparison: replaying demonstrations on near-duplicate test scenes (0 of 14 twin-test episodes succeeded, mostly from grasp misses), time-compressed action playback, gripper-correction add-ons for the Comet family, sampled action search, a modified robot configuration, 224-pixel camera inputs, and a third-party 100-task model whose port to our serving stack never produced enough nonzero development rollouts to rule out a porting error. Given the power analysis above, "rejected" often means "not shown to help" rather than "shown not to help."

9 Serving for hidden evaluation

Hidden evaluation runs the submitted containers on the organizers' hardware. We package one image per routed policy, and each container is bound at runtime to a single task. The server enforces a strict contract on every frame and fails closed:

- one task per container and one evaluator session at a time (a second connection is refused with close code 1013);
- batches of 1–10 environments with a fixed shape until reset; a reset clears the action queues before the next action;
- a single global transport chunk $K = 4$ that divides both native replay lengths (32 and 20), so the chunked action stream is identical to the native receding-horizon stream (tested for both lengths);
- malformed frames, wrong shapes, nonfinite actions, model exceptions and per-query timeouts (550 s, below the 600 s rule) close the connection with code 1011 and stop the process; there is no retry, no substitute

action, no fallback checkpoint and no automatic restart, because a fabricated action could turn a fault into a silent wrong result.

The release contains this layer (codec, contract, chunk executor, server and client) with integration tests against a scripted policy. In the original packaging, all ten native models passed startup, warm-up and synthetic-input batch-1 and batch-10 protocol tests on an RTX 3090 as an unprivileged user; the longest batch-10 query took 3.38 s and sampled GPU memory stayed below 9 GB. A separate strict-isolation test, which restricted file writes and outbound connections, failed for all ten because CUDA reported no visible GPU. We have not established whether the isolation harness or the images cause this. We have not completed full simulator episodes under Isaac Sim 5.1 at the required one frame per second on the organizers' GPU class, so hidden-evaluation readiness remains unverified.

10 Limitations and disclosures

- Q is a self-evaluated score on public reporting instances produced with Isaac Sim 6.0.1, not a hidden-test score or a prediction of placement. Hidden evaluation uses Isaac Sim 5.1, and simulator differences may shift scores.
- Development evidence is thin and concentrated on a few instances. Routes for individual tasks are therefore uncertain even when the aggregate ranking is informative.
- Two duplicate cells were resolved by completion order, and seven episodes used a different replay length. Both are disclosed above and left unchanged.
- We do not redistribute model weights. The publishers of the winner checkpoints and the four-task specialist declare Apache-2.0 at the pinned revisions. The ymetz card defers to its base checkpoint's terms. We did not find an explicit licence for the separately hosted Comet weights or for the specialist used on tasks 96–97. Credits are attribution, not a licence grant.
- The released selector reproduces the frozen stage-2 decisions closely but not exactly, and the stage-1 priors depend on reference tables we do not redistribute.

11 Conclusion

A bank of public checkpoints, routed per task from development data alone, reached $Q = 0.156$ on the public reporting instances of the 2026 BEHAVIOR Challenge. Most of the score comes from partial credit, and a third of the tasks remain out of reach for every checkpoint we could run. The more transferable lesson concerns measurement. At ten-instance scale with per-rollout noise of 0.20, the best-looking choice is reliably optimistic and per-task gains below about 0.1 cannot be verified. Priors that borrow strength across tasks and policies, conservative switching rules, a frozen route, and score-blind bookkeeping are what make the reported number trustworthy. Code, data and the route are available at https://github.com/FungousLand1941/policy-bank.

Acknowledgments

We thank the authors of every checkpoint in the bank: Team Comet, Ilia Larchenko, Gleb Zarin and Akash Karnatak, ymetz, JackLiu0406 and amyjia1212. We also thank Physical Intelligence for OpenPI and Google for PaliGemma, the Stanford BEHAVIOR-1K team for the benchmark, the challenge and the demonstrations, and the Texas Advanced Computing Center for Stampede3. Parts of the evaluation tooling and of this report's analysis code were written with AI coding assistants. Every number in this report is recomputed from the released data by the released code.

References

[1] Junjie Bai, Yu-Wei Chao, Qizhi Chen, Jinwei Gu, Moo Jin Kim, Zhaoshuo Li, Xuan Li, Tsung-Yi Lin, Ming-Yu Liu, Nic Ma, Kaichun Mo, Delin Qu, Shangkun Sun, Hongchi Xia, Fangyin Wei, and Xiaohui Zeng. Openpi Comet: Competition solution for 2025 BEHAVIOR challenge. *arXiv preprint arXiv:2512.10071*, 2025.

[2] Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, et al. PaliGemma: A versatile 3B VLM for transfer. *arXiv preprint arXiv:2407.07726*, 2024.

[3] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[4] Bradley Efron and Carl Morris. Data analysis using Stein's estimator and its generalizations. *Journal of the American Statistical Association*, 70(350):311–319, 1975.

[5] Ilia Larchenko, Gleb Zarin, and Akash Karnatak. Task adaptation of vision-language-action model: 1st place solution for the 2025 BEHAVIOR challenge. *arXiv preprint arXiv:2512.06951*, 2025.

[6] Ilia Larchenko, Gleb Zarin, and Akash Karnatak. behavior_submission: 2025 BEHAVIOR challenge winning checkpoints. https://huggingface.co/IliaLarchenko/behavior_submission, 2025.

[7] Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, et al. BEHAVIOR-1K: A benchmark for embodied AI with 1,000 everyday activities and realistic simulation. In *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, 2023.

[8] Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, et al. BEHAVIOR-1K: A human-centered, embodied AI benchmark with 1,000 everyday activities and realistic simulation. *arXiv preprint arXiv:2403.09227*, 2024.

[9] Physical Intelligence. openpi: Open-source models and code for robot learning. https://github.com/Physical-Intelligence/openpi, 2025.

[10] Physical Intelligence, Kevin Black, Noah Brown, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[11] John R. Rice. The algorithm selection problem. In *Advances in Computers*, volume 15, pages 65–118. Elsevier, 1976.

[12] James E. Smith and Robert L. Winkler. The optimizer's curse: Skepticism and postdecision surprise in decision analysis. *Management Science*, 52(3):311–322, 2006.

[13] Stanford Vision and Learning Lab. The 2026 BEHAVIOR challenge. https://behavior.stanford.edu/challenge/, 2026. Rules, evaluation protocol and submission guidelines; accessed October 2026.

[14] Team Comet. openpi_comet model weights. https://huggingface.co/sunshk/openpi_comet, 2025.